Bài tập Toán 03–04 — Ma trận, hệ tuyến tính và PCA

1. Đề bài tiếng Anh

1. Data Exploration and Matrix Creation: create an eight-row store matrix with Orders, Customers, Revenue, and Employees; display matrix shape, customer values, and average customers.
2. Finding Evaluation Weights Using a Linear System: solve the supplied 3 × 3 system A @ w = b for three store-evaluation weights.
3. Calculating and Ranking Store Performance: scale Orders, Customers and Revenue into comparable score units, compute S @ w, rank all eight stores and identify the best store.
4. Data Standardization and Dimensionality Reduction Using PCA: standardize the four store features, apply PCA to reduce to two components, and inspect projected shape and explained variance.

2. Đề bài tiếng Việt

1. Khám phá dữ liệu và tạo ma trận: lập bảng tám cửa hàng với Đơn hàng, Khách hàng, Doanh thu và Nhân viên; in kích thước ma trận, cột Khách hàng và số khách trung bình.
2. Tìm trọng số đánh giá bằng hệ tuyến tính: giải hệ 3 × 3 đã cho theo biểu thức A @ w = b để tính ba trọng số đánh giá.
3. Tính và xếp hạng hoạt động cửa hàng: quy đổi Đơn hàng, Khách hàng và Doanh thu thành các thang điểm tương ứng, tính S @ w, xếp tám cửa hàng và tìm cửa hàng cao điểm nhất.
4. Chuẩn hóa dữ liệu và giảm số chiều bằng PCA: chuẩn hóa bốn cột dữ liệu, đưa về hai thành phần chính, kiểm tra kích thước sau biến đổi và phần trăm phương sai được giữ lại.

3. Code Python có chú thích

```
"""Lab Math 03-04 — store evaluation using matrices, linear systems and PCA.

Adapted from 2. Math/labs3_4.ipynb. Run:
    python SOLUTION_ANNOTATED.py

Requires numpy and scikit-learn; prints results without opening a GUI.
"""
import numpy as np  # Arrays, descriptive statistics and matrix multiplication.
from sklearn.preprocessing import StandardScaler  # Scale each feature.
from sklearn.decomposition import PCA  # Reduce correlated features to 2 axes.

# Rows A-H represent stores. Each column has a specified real-world unit:
# Orders (count), Customers (count), Revenue (currency units), Employees (count).
STORE_NAMES = np.array(list("ABCDEFGH"))  # Gán kết quả của biểu thức vào biến để sử dụng ở bước tiếp theo.
STORE_DATA = np.array([
    [300, 400, 100000, 20],
    [400, 500, 140000, 25],
    [200, 200, 70000, 15],
    [500, 600, 180000, 30],
    [350, 450, 115000, 22],
    [450, 550, 160000, 28],
    [250, 300, 85000, 18],
    [600, 700, 210000, 35],
], dtype=float)  # Shape (8,4), with numeric values compatible with NumPy.
```

```
# A and b are supplied evaluation coefficients and observed benchmark scores.
EQUATION_MATRIX = np.array([[3, 4, 10], [4, 5, 14], [2, 2, 7]], dtype=float)  # Gán kết quả của biểu thức vào biến để sử dụng ở bước tiếp theo.
EQUATION_SCORES = np.array([135, 182, 86], dtype=float)  # Gán kết quả của biểu thức vào biến để sử dụng ở bước tiếp theo.


def task1_explore(stores=STORE_DATA):  # Định nghĩa một hàm có thể gọi lại; các tham số nằm trong dấu ngoặc.
    """Inspect matrix dimensions, customer counts and mean customer traffic."""
    if stores.ndim != 2 or stores.shape[1] != 4:  # Nếu điều kiện đúng, chạy khối lệnh thụt vào phía dưới.
        raise ValueError("Expect a matrix with four store features.")  # Báo lỗi nếu dữ liệu không đúng điều kiện.
    customers = stores[:, 1]  # ':' means all rows; index 1 is Customers.
    mean_customers = float(np.mean(customers))  # Mean over all eight stores.
    return stores.shape, customers, mean_customers  # Trả kết quả về nơi gọi hàm.


def task2_weights(A=EQUATION_MATRIX, b=EQUATION_SCORES):  # Định nghĩa một hàm có thể gọi lại; các tham số nằm trong dấu ngoặc.
    """Solve A @ w = b for weights of Orders, Customers and Revenue."""
    # np.linalg.solve avoids manually computing the matrix inverse.
    if A.shape != (3, 3) or b.shape != (3,):  # Nếu điều kiện đúng, chạy khối lệnh thụt vào phía dưới.
        raise ValueError("Expected a 3x3 matrix and a three-element score vector.")  # Báo lỗi nếu dữ liệu không đúng điều kiện.
    if np.linalg.matrix_rank(A) < 3:  # A singular matrix has no unique answer.
        raise ValueError("The benchmark system has no unique solution.")  # Báo lỗi nếu dữ liệu không đúng điều kiện.
    weights = np.linalg.solve(A, b)  # Gán kết quả của biểu thức vào biến để sử dụng ở bước tiếp theo.
    if not (np.allclose(A @ weights, b)): raise ValueError("Kiểm tra dữ liệu không đạt")  # Reconstruct the input equations.
    return weights  # Trả kết quả về nơi gọi hàm.


def task3_rank(stores=STORE_DATA, names=STORE_NAMES, weights=None):  # Định nghĩa một hàm có thể gọi lại; các tham số nằm trong dấu ngoặc.
    """Build dimensionless components, combine weights, rank all stores."""
    if weights is None:  # Nếu điều kiện đúng, chạy khối lệnh thụt vào phía dưới.
        weights = task2_weights()  # Gán kết quả của biểu thức vào biến để sử dụng ở bước tiếp theo.
    # Scale columns as in the original notebook before applying weights.
    components = np.column_stack((
        stores[:, 0] / 100,  # One score unit per 100 orders.
        stores[:, 1] / 100,  # One score unit per 100 customers.
        stores[:, 2] / 10000,  # One score unit per 10,000 revenue units.
    ))
    totals = components @ weights  # (8x3) @ (3,) -> eight store scores.
    ranked = sorted(zip(names.tolist(), totals.tolist()),
                    key=lambda pair: (-pair[1], pair[0]))
    winner_index = int(np.argmax(totals))  # Index of highest score.
    return components, totals, ranked, names[winner_index]  # Trả kết quả về nơi gọi hàm.


def task4_pca(stores=STORE_DATA):  # Định nghĩa một hàm có thể gọi lại; các tham số nằm trong dấu ngoặc.
    """Standardize four columns, then project stores onto two PCA axes."""
    # Different units (currency, people) must not bias distance by magnitude.
    scaler = StandardScaler()  # By default centers and scales each column.
    standardized = scaler.fit_transform(stores)  # Gán kết quả của biểu thức vào biến để sử dụng ở bước tiếp theo.
    # PCA finds directions containing the largest variation in scaled data.
    pca = PCA(n_components=2)  # We want a shape of (8 stores, 2 axes).
    projected = pca.fit_transform(standardized)  # Gán kết quả của biểu thức vào biến để sử dụng ở bước tiếp theo.
    retained = pca.explained_variance_ratio_  # Fraction on each retained axis.
    if not (projected.shape == (stores.shape[0], 2)): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra điều kiện bằng if thay cho assert.
    if not (np.allclose(standardized.mean(axis=0), 0, atol=1e-12)): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra điều kiện bằng if thay cho assert.
    if not (np.allclose(standardized.var(axis=0), 1, atol=1e-12)): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra điều kiện bằng if thay cho assert.
    if not (np.isfinite(projected).all()): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra điều kiện bằng if thay cho assert.
    return standardized, projected, retained  # Trả kết quả về nơi gọi hàm.
```

```
def main():  # Định nghĩa một hàm có thể gọi lại; các tham số nằm trong dấu ngoặc.
    """Print intermediate results for every English task and check shapes."""
    shape, customers, mean_customers = task1_explore()
    print("Task 1 - Store matrix shape:", shape)  # In kết quả để đối chiếu với đề bài.
    print("Customers:", customers.astype(int).tolist())  # In kết quả để đối chiếu với đề bài.
    print("Mean customers:", round(mean_customers, 2))  # In kết quả để đối chiếu với đề bài.

    weights = task2_weights()  # Gán kết quả của biểu thức vào biến để sử dụng ở bước tiếp theo.
    print("Task 2 - Weights:", np.round(weights, 6).tolist())  # In kết quả để đối chiếu với đề bài.

    components, scores, ranking, best = task3_rank(weights=weights)
    print("Task 3 - Component matrix shape:", components.shape)  # In kết quả để đối chiếu với đề bài.
    print("Scores:", np.round(scores, 2).tolist())  # In kết quả để đối chiếu với đề bài.
    print("Ranking:", [(name, round(score, 2)) for name, score in ranking])  # In kết quả để đối chiếu với đề bài.
    print("Best store:", best)  # In kết quả để đối chiếu với đề bài.

    standardized, projection, ratios = task4_pca()
    print("Task 4 - Standardized shape:", standardized.shape)  # In kết quả để đối chiếu với đề bài.
    print("PCA projected shape:", projection.shape)  # In kết quả để đối chiếu với đề bài.
    print("Explained variance fractions:", np.round(ratios, 4).tolist())  # In kết quả để đối chiếu với đề bài.
    print("Total variance retained:", f"{ratios.sum() * 100:.2f}%")  # In kết quả để đối chiếu với đề bài.
    if not (len(ranking) == len(STORE_NAMES)): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra điều kiện bằng if thay cho assert.
    if not (projection.shape == (8, 2)): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra điều kiện bằng if thay cho assert.


if __name__ == "__main__":  # Nếu điều kiện đúng, chạy khối lệnh thụt vào phía dưới.
    main()  # Execute this demonstration only when launched as a script.
```

4. Giải thích nội dung và cách giải

Câu 1

`np.array` tạo ma trận có tám hàng và bốn cột nên `shape` là `(8, 4)`. `stores[:, 1]` có nghĩa là lấy **mọi hàng**, riêng cột chỉ số 1 (Customers); `np.mean` tính trung bình tám số khách.

Câu 2

`np.linalg.solve(A, b)` giải **hệ phương trình** `A @ w = b`, không phải nhân từng phần tử. Ma trận 3 × 3 chỉ có nghiệm duy nhất nếu đủ hạng; `np.linalg.matrix_rank(A)` kiểm tra điều này.

Câu 3

`np.column_stack` ghép ba cột đã chia thang 100, 100, 10000; `components @ weights` nhân ma trận 8 × 3 với vector 3 phần tử tạo tám điểm. `np.argmax` cho vị trí điểm lớn nhất; `sorted` sắp xếp cửa hàng.

Câu 4

Doanh thu tính bằng tiền lớn hơn hẳn số nhân viên, nên cần `StandardScaler()` để đưa các đặc trưng về thang đo có trung bình gần 0 và phương sai gần 1. `PCA(n_components=2)` biến dữ liệu tám hàng bốn cột thành tám hàng hai cột; `explained_variance_ratio_` cho biết tỷ lệ phương sai mỗi thành phần giữ lại. PCA không bảo đảm dự đoán hoàn hảo.

Đối chiếu nguồn và hướng dẫn chạy

Notebook PythoClass gốc có bốn tiêu đề đề bài tiếng Anh và mã minh họa. Lời giải đầy đủ bên dưới lấy từ `labs/pythoclass/LAB-MATH-03-04-matrix-pca/SOLUTION.py`, bản tái cấu trúc từ notebook, vẫn sử dụng dữ liệu 8 cửa hàng và hệ phương trình gốc.

Khác biệt trình bày: Các điều kiện `assert` có trong `SOLUTION.py` được đổi thành `if not ...: raise ValueError(...)` để người mới hiểu cách kiểm tra rõ ràng hơn; không đổi mục tiêu tính toán.

Nguồn bài tập: `PythoClass/2. Math/labs3_4.ipynb` trên repo `nguyenletrian/PythoClass`.